

深圳大学实验报告

课程名称： 计算机网络

实验项目名称： 实验3 数据包抓取与分析

学院： 计算机与软件学院

专业： 计算机科学与技术

指导教师： 王璐

报告人： 姚泽棉 学号： 2024150026 班级： 高性能班

实验时间： 2026/04/01

实验报告提交时间： 2026/04/01

教务处制

实验目的

学习安装、使用协议分析软件，掌握基本的数据报抓取、过滤和分析方法，能分析 HTTP、TCP、ICMP 等协议。

实验环境

1. 使用具有 Internet 连接的 Windows 操作系统；
2. 抓包软件 Wireshark。

实验内容：

3. 安装学习 Wireshark 软件；
4. 抓包与分析 HTTP 协议；
5. 分析 TCP 协议；
6. 分析 TCP 三次握手；
7. 分析 ICMP 协议。

实验步骤：

1 安装学习 Wireshark 软件

Wireshark 是世界上最广泛使用的网络协议分析器，下载官网地址：<https://www.wireshark.org/>。软件使用手册：<https://www.wireshark.org/download/docs/user-guide.pdf>。

1.1 运行软件并捕获分组

运行 Wireshark，初始界面如图 1.1.1 所示。从接口列表中选择要捕获的接口，双击即可开始捕获。（实验时根据哪个网络有后面的流量，选择具体捕获哪个接口）

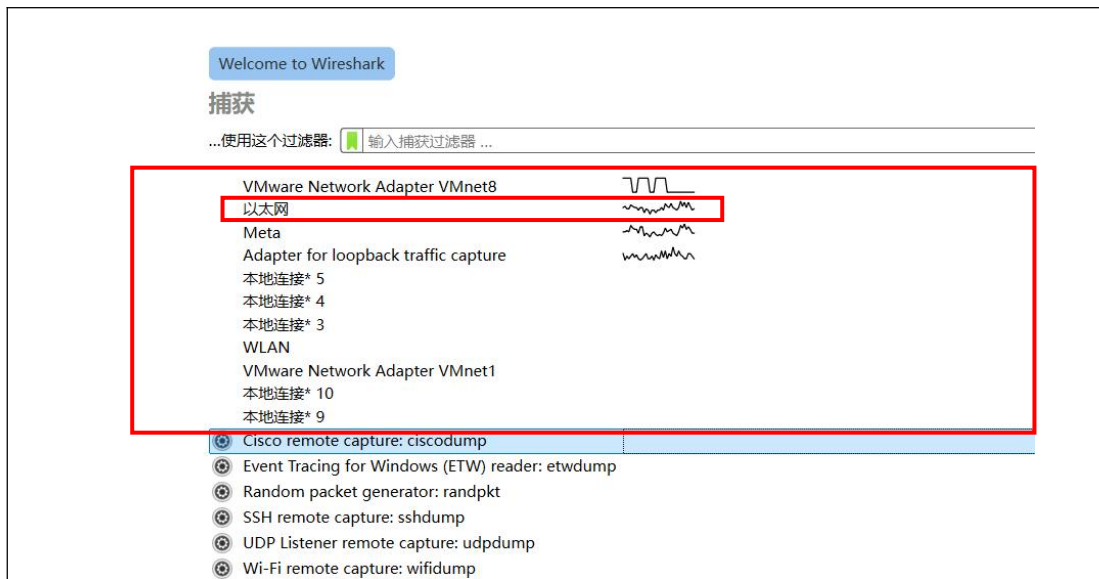


图 1.1.1 Wireshark 的主界面

选择以太网并开始捕获分组，捕获分组的状态如图 1.1.2 所示。

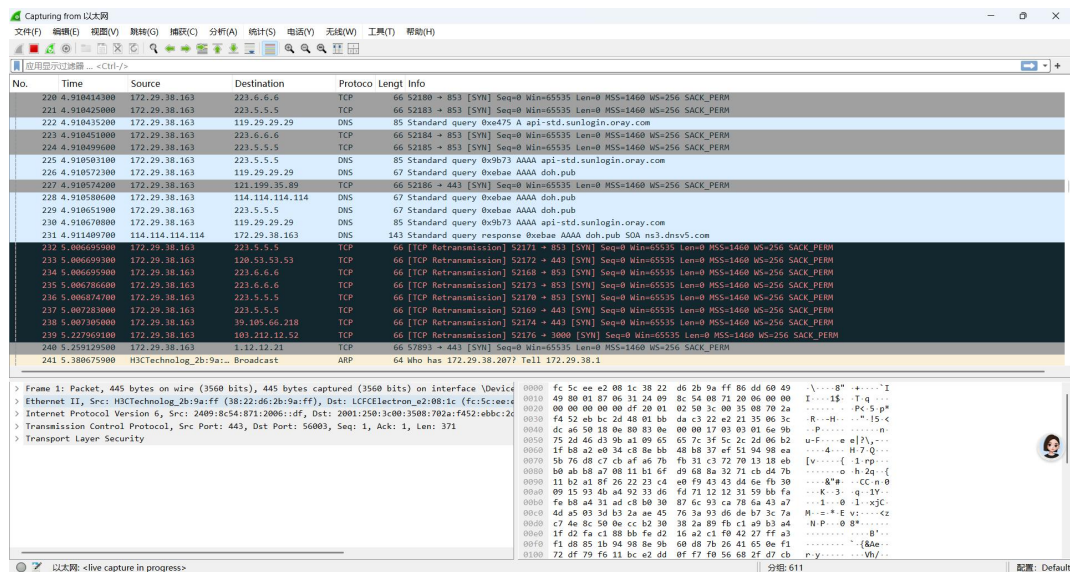


图 1.1.2 Wireshark 捕获分组

1.2 学习使用过滤器

过滤包括协议过滤、IP 地址过滤、模式过滤和端口过滤等。

协议过滤以 HTTP 为例，捕获结果如图 1.2.1 所示：

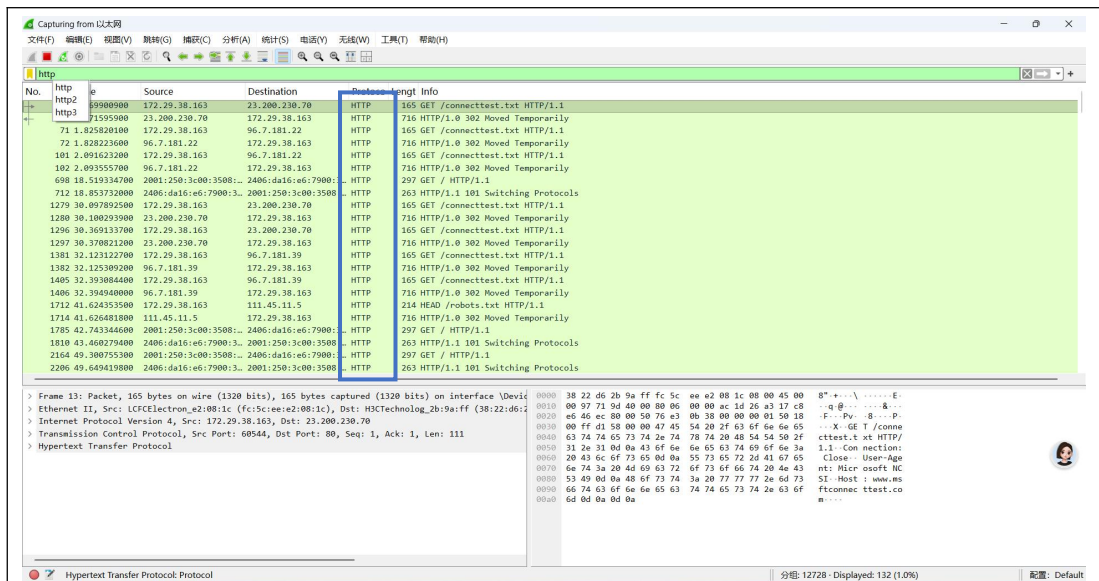


图 1.2.1 HTTP 协议过滤

IP 地址过滤以 `ip.src == 172.29.38.163 and ip.dst == 23.200.230.70` 为例，含义为过滤源地址是 172.29.38.163 并且目的地址是 23.200.230.70 的分组。捕获的结果如图 1.2.2 所示。

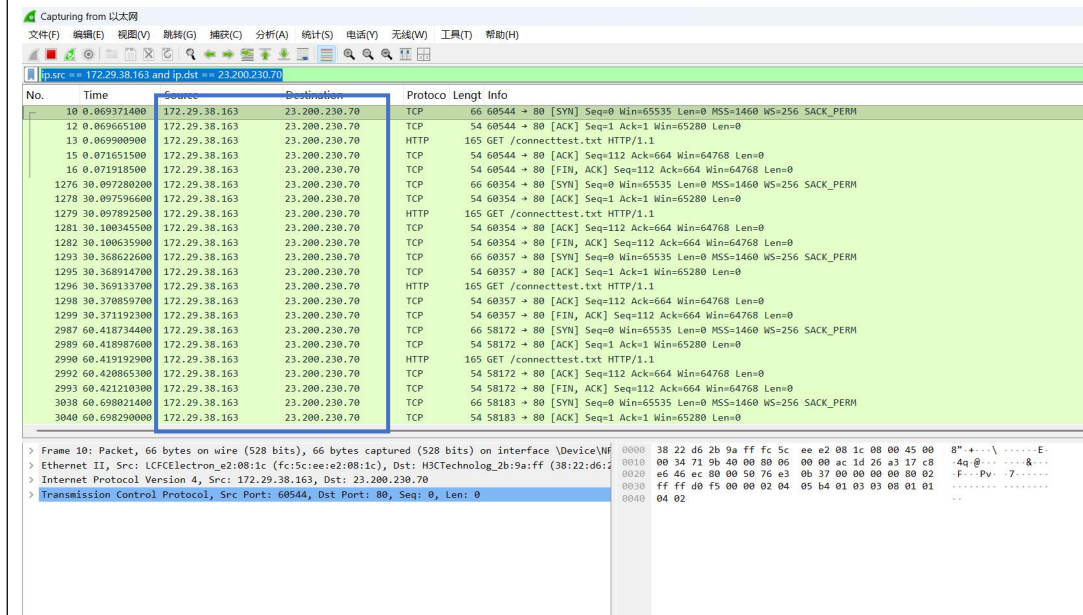


图 1.2.2 IP 地址过滤

模式过滤以 `http.request.method == "GET"` 为例，含义为过滤 http 请求方法是 GET 的分组。结果如图 1.2.3 所示；除了 GET 方法外，还有其他方法如 POST 方法，如图 1.2.4 所示。

Capturing from 以太网

文件(F) 编辑(E) 视图(V) 跳转(G) 捕获(C) 分析(A) 统计(S) 电话(Y) 无线(W) 工具(T) 帮助(H)

http.request.method=="GET"

No.	Time	Source	Destination	Protocol	Length	Info
13	0.069900900	172.29.38.163	23.200.230.70	HTTP	165	GET /connecttest.txt HTTP/1.1
71	1.825820100	172.29.38.163	96.7.181.22	HTTP	165	GET /connecttest.txt HTTP/1.1
101	2.091623200	172.29.38.163	96.7.181.22	HTTP	165	GET /connecttest.txt HTTP/1.1
698	18.519334700	2001:250:3c00:3508::...	2406:da16:e6:7900:3...	HTTP	297	GET / HTTP/1.1
1279	30.097892500	172.29.38.163	23.200.230.70	HTTP	165	GET /connecttest.txt HTTP/1.1
1296	30.369133700	172.29.38.163	23.200.230.70	HTTP	165	GET /connecttest.txt HTTP/1.1
1381	32.123122700	172.29.38.163	96.7.181.39	HTTP	165	GET /connecttest.txt HTTP/1.1
1405	32.393084400	172.29.38.163	96.7.181.39	HTTP	165	GET /connecttest.txt HTTP/1.1
1785	42.743344600	2001:250:3c00:3508::...	2406:da16:e6:7900:3...	HTTP	297	GET / HTTP/1.1
2164	49.300755300	2001:250:3c00:3508::...	2406:da16:e6:7900:3...	HTTP	297	GET / HTTP/1.1
2491	54.397248100	2001:250:3c00:3508::...	2406:da16:e6:7900:3...	HTTP	297	GET / HTTP/1.1
2825	58.830721500	172.29.38.163	172.30.255.42	HTTP	202	GET /a79.htm?wlanuserip=172.29.38.163&
2880	59.554438300	172.29.38.163	172.30.255.42	HTTP	202	GET /a79.htm?wlanuserip=172.29.38.163&
2898	59.670811600	172.29.38.163	172.30.255.42	HTTP	202	GET /a79.htm?wlanuserip=172.29.38.163&
2972	60.373323700	172.29.38.163	172.30.255.42	HTTP	202	GET /a79.htm?wlanuserip=172.29.38.163&
2990	60.419192900	172.29.38.163	23.200.230.70	HTTP	165	GET /connecttest.txt HTTP/1.1
2998	60.435444700	2001:250:3c00:3508::...	2406:da16:e6:7900:3...	HTTP	297	GET / HTTP/1.1
3041	60.698506100	172.29.38.163	23.200.230.70	HTTP	165	GET /connecttest.txt HTTP/1.1
3068	60.901885300	172.29.38.163	120.233.47.200	HTTP	233	GET /params/login_params_tcp.json HTTP/
3167	62.422158900	172.29.38.163	23.206.188.212	HTTP	165	GET /connecttest.txt HTTP/1.1
3205	62.688951500	172.29.38.163	23.206.188.212	HTTP	165	GET /connecttest.txt HTTP/1.1
3558	68.787237600	2001:250:3c00:3508::...	2406:da16:e6:7900:3...	HTTP	297	GET / HTTP/1.1

> Frame 13: Packet, 165 bytes on wire (1320 bits), 165 bytes captured (10936 bits) on interface \Device\NPF{...} Ethernet II, Src: LCFCElectron_e2:08:1c (fc:5c:ee:e2:08:1c), Dst: H3CTechnolog_2b:9a:ff (38:22:d6:2b:9a:ff) Internet Protocol Version 4, Src: 172.29.38.163, Dst: 23.200.230.70 Transmission Control Protocol, Src Port: 60544, Dst Port: 80, Seq: 1, Ack: 1, Len: 111 Hypertext Transfer Protocol

0000 38 22 d6
0010 00 97 71
0020 e6 46 ec
0030 00 ff d1
0040 63 74 74
0050 31 2e 31

图 1.2.3 模式过滤 (GET)

Capturing from 以太网

文件(F) 编辑(E) 视图(V) 跳转(G) 捕获(C) 分析(A) 统计(S) 电话(Y) 无线(W) 工具(T) 帮助(H)

http.request.method=="POST"

No.	Time	Source	Destination	Protocol	Length	Info
2817	58.826434400	172.29.38.163	124.236.26.172	HTTP/1.1	1367	POST / HTTP/1.1, JSON (application/json)
2875	59.551389600	172.29.38.163	124.236.26.172	HTTP/1.1	1367	POST / HTTP/1.1, JSON (application/json)
2894	59.668577400	172.29.38.163	124.236.26.172	HTTP/1.1	1367	POST / HTTP/1.1, JSON (application/json)
2967	60.370405400	172.29.38.163	124.236.26.172	HTTP/1.1	1367	POST / HTTP/1.1, JSON (application/json)
4186	83.755297900	172.29.38.163	124.236.26.172	HTTP/1.1	1308	POST / HTTP/1.1, JSON (application/json)
4428	87.981242500	172.29.38.163	124.236.26.172	HTTP/1.1	1347	POST / HTTP/1.1, JSON (application/json)
9083	123.006037400	172.29.38.163	124.236.26.172	HTTP/1.1	1347	POST / HTTP/1.1, JSON (application/json)
11214	158.036874700	172.29.38.163	124.236.26.172	HTTP/1.1	1347	POST / HTTP/1.1, JSON (application/json)
12802	193.045192700	172.29.38.163	124.236.26.172	HTTP/1.1	1347	POST / HTTP/1.1, JSON (application/json)
13435	208.914722300	2001:250:3c00:3508::...	2402:4e00:1020:2415::...	HTTP	1087	POST /mntls/0000797d HTTP/1.1
13438	208.921952300	2001:250:3c00:3508::...	2402:4e00:1430:1f29::...	HTTP	1079	POST /mntls/0000797d HTTP/1.1
14839	241.517592300	172.29.38.163	124.236.26.172	HTTP/1.1	1023	POST / HTTP/1.1, JSON (application/json)
15611	258.633153600	172.29.38.163	124.236.26.172	HTTP/1.1	1172	POST / HTTP/1.1, JSON (application/json)
15821	264.042776300	172.29.38.163	124.236.26.172	HTTP/1.1	1202	POST / HTTP/1.1, JSON (application/json)
15868	264.512793400	2001:250:3c00:3508::...	2402:4e00:1020:2415::...	HTTP	845	POST /mntls/00007a30 HTTP/1.1
16333	272.793525200	2001:250:3c00:3508::...	2402:4e00:1020:2415::...	HTTP	869	POST /mntls/00007a4e HTTP/1.1
17523	299.045923600	172.29.38.163	124.236.26.172	HTTP/1.1	1202	POST / HTTP/1.1, JSON (application/json)
18809	324.182886200	2001:250:3c00:3508::...	2402:4e00:1020:2415::...	HTTP	1063	POST /mntls/00007af4 HTTP/1.1
18840	324.576862500	2001:250:3c00:3508::...	2402:4e00:1020:2415::...	HTTP	853	POST /mntls/00007af7 HTTP/1.1
19315	334.076190700	172.29.38.163	124.236.26.172	HTTP/1.1	1202	POST / HTTP/1.1, JSON (application/json)
20547	349.583636900	2001:250:3c00:3508::...	2402:4e00:1020:2415::...	HTTP	1029	POST /mntls/00007b49 HTTP/1.1
20554	349.612919900	2001:250:3c00:3508::...	2409:8c00:8401:1000::...	HTTP	1042	POST /mntls/00007b49 HTTP/1.1

> Frame 2817: Packet, 1367 bytes on wire (10936 bits), 1367 bytes captured (10936 bits) on interface \Device\NPF{...} Ethernet II, Src: LCFCElectron_e2:08:1c (fc:5c:ee:e2:08:1c), Dst: H3CTechnolog_2b:9a:ff (38:22:d6:2b:9a:ff) Internet Protocol Version 4, Src: 172.29.38.163, Dst: 124.236.26.172

0000 38 22 d6 2b 9a ff
0010 05 49 7a c5 40 00
0020 1a ac e7 42 00 50
0030 00 ff 6f 94 00 00

图 1.2.4 模式过滤 (POST)

端口过滤以 `tcp.port == 80` 为例，含义为过滤 TCP 端口号是 80 的分组，如图 1.2.5、1.2.6 所示。这里只要端口为 80 就会筛选，包括了 Src 和 Dst。

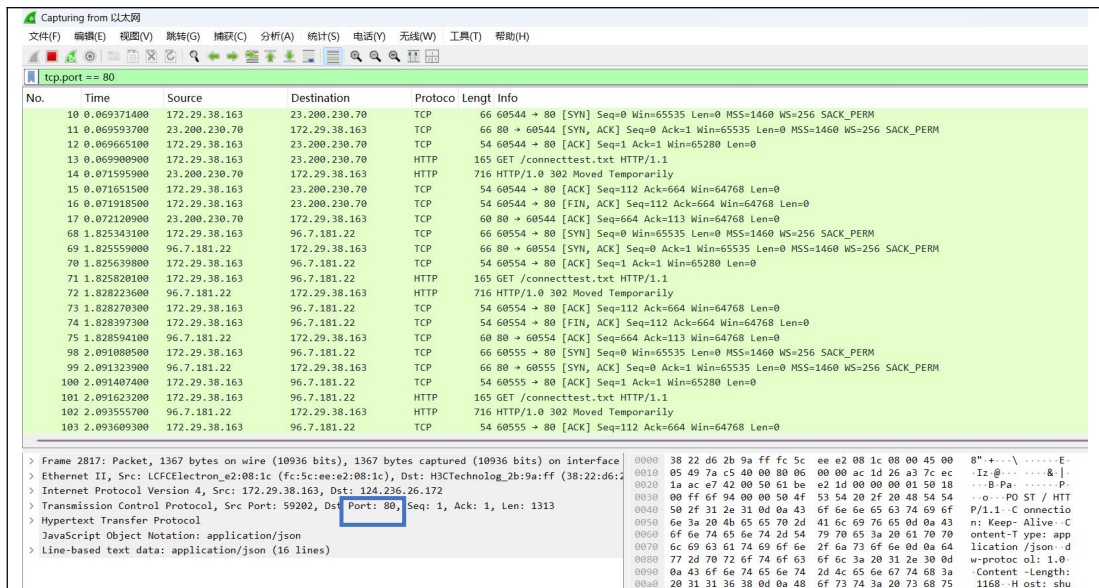


图 1.2.5 端口过滤 (tcp.port==80, 该图为 Dst)

2 抓包与分析 HTTP 协议

2.1 开启 Wireshark 抓包

开启 Wireshark 抓包，在过滤器中输入 http，即过滤 http 协议的分组；

接下来打开浏览器，输入一个网址，以 SZUOJ: 172.31.221.67 为例；Wireshark 返回的结果如图 2.1.1 所示，可以看到其分组列表栏中出现了 HTTP 协议分组。

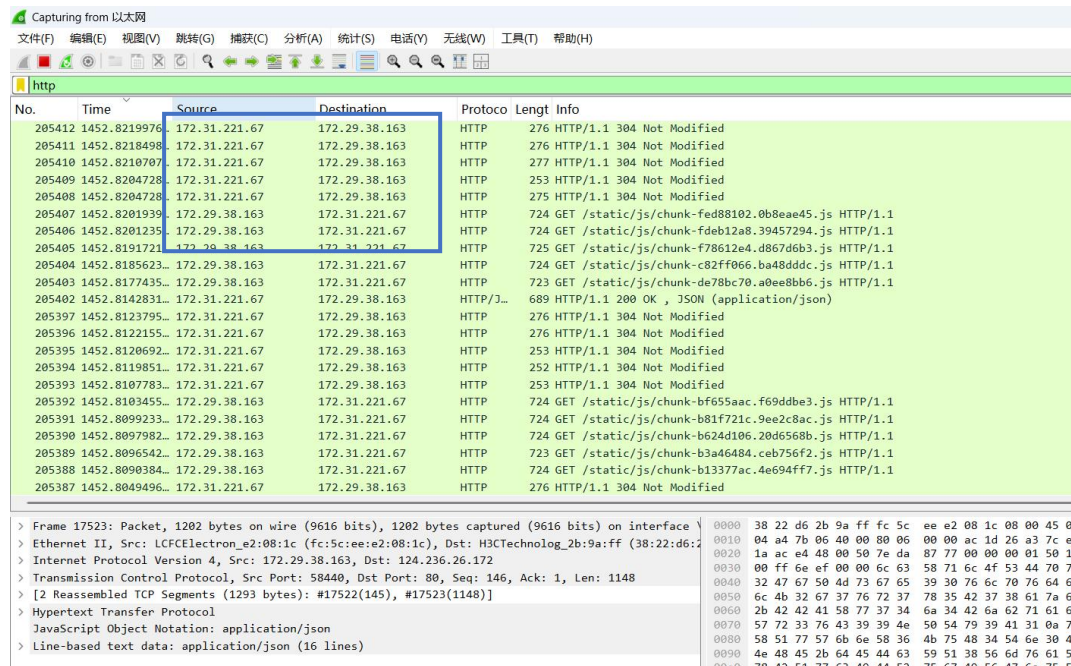


图 2.1.1 观察到 Wireshark 分组列表栏中出现 HTTP 协议分组

2.2 分析 HTTP 协议

分析哪些分组是前一步浏览网页发生可以根据 IP 地址来判断。当 IP 中包含 172.31.221.67 时，该分组就是由于前一步打开 OJ 网页而产生；单击任意分组，分组详情栏显示其具体信息。



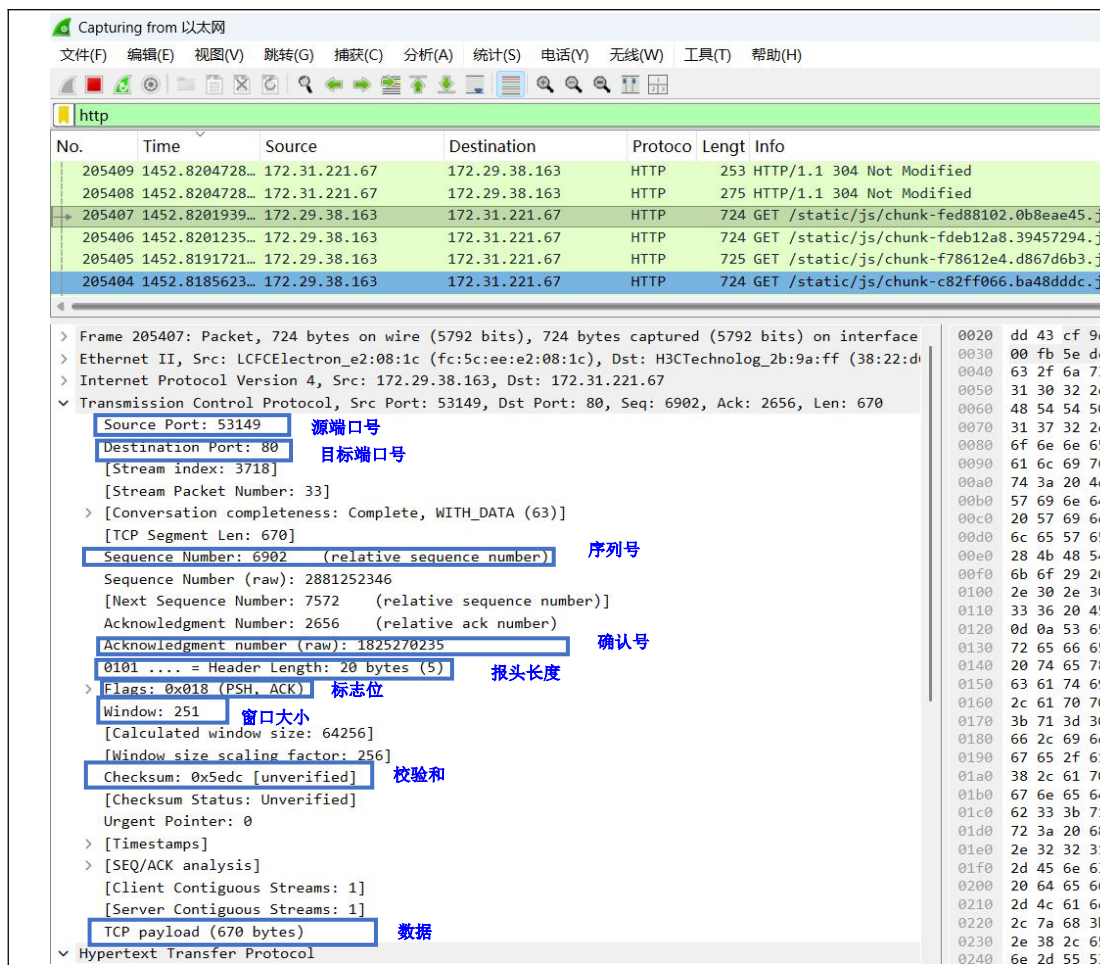


图 3.1.1 分析 HTTP 协议传输中的 TCP 协议信息

3.2 追踪分组的 TCP 流

对图 3.1.1 的分组，追踪其 TCP 流。点击右键，从下拉菜单中选择 TCP 流，如图 3.2.1 所示。在选择后上方的筛选条件将变为 tcp.stream eq 331；

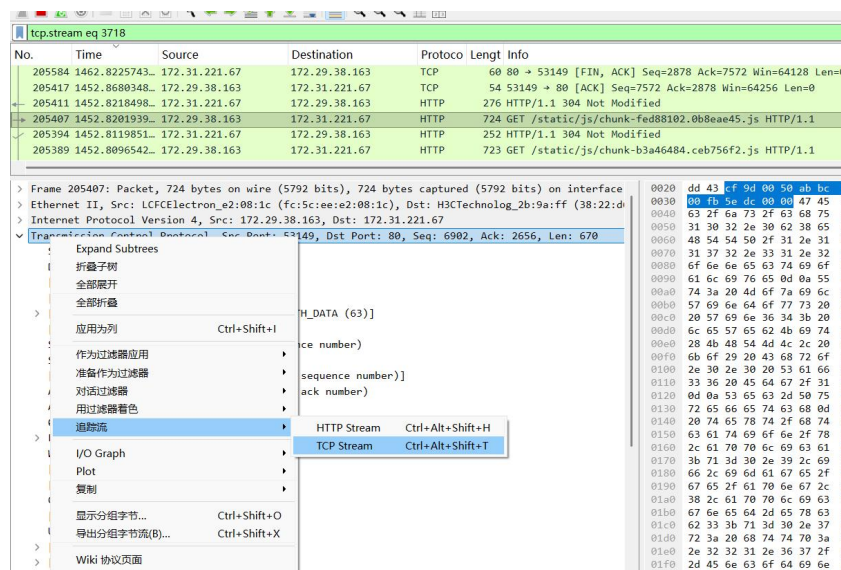


图 3.2.1 右击分组追踪 TCP 流

接下来就可以去寻找 TCP 建立连接的分组。根据原理：1)TCP 连接建立应该在 HTTP GET 请求之前完成；2) TCP 建立连接时会设置标志位 SYN（即第一次握手：客户端将同步号 SYN 置为 1，随机产生一个值 $seq=x$ ，将该数据包发送给服务端，客户端进入 SYN_SENT 状态，等待服务端确认）。因此筛选后按时间顺序，滑动到最上方即可找到建立连接的分组，如图 3.2.2 所示。

No.	Time	Source	Destination	Protocol	Length	Info
205094	1452.271988...	172.29.38.163	172.31.221.67	TCP	66	53149 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
205102	1452.2732488...	172.31.221.67	172.29.38.163	TCP	66	80 → 53149 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1456 SACK_PERM WS=
205103	1452.2732839...	172.29.38.163	172.31.221.67	TCP	54	53149 → 80 [ACK] Seq=1 Ack=1 Win=65280 Len=0
205105	1452.2734394...	172.29.38.163	172.31.221.67	HTTP	556	GET /tinytce/tinytce.min.js HTTP/1.1
205114	1452.2756600...	172.31.221.67	172.29.38.163	TCP	60	80 → 53149 [ACK] Seq=1 Ack=503 Win=64128 Len=0
205116	1452.2761291...	172.31.221.67	172.29.38.163	HTTP	277	HTTP/1.1 304 Not Modified
205136	1452.3161878...	172.29.38.163	172.31.221.67	TCP	54	53149 → 80 [ACK] Seq=503 Ack=224 Win=65280 Len=0
205163	1452.5482477...	172.29.38.163	172.31.221.67	HTTP	725	GET /static/css/chunk-04a9f3fa.76ac9832.css HTTP/1.1

图 3.2.2 找到 TCP 建立连接的分组

4 分析 TCP 三次握手

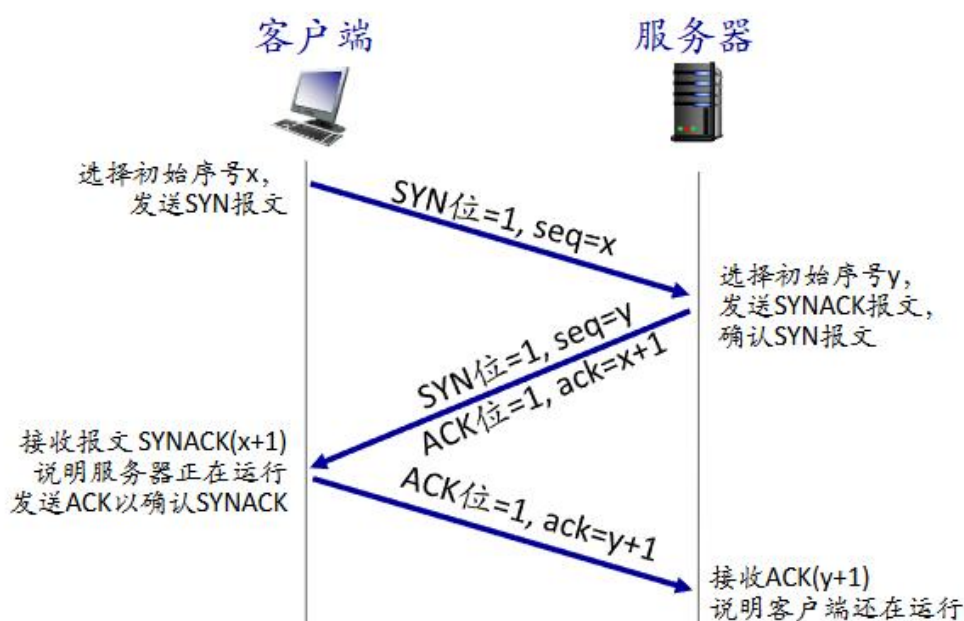
TCP 三次握手建立连接和数据传输的过程：

第一次握手：客户端将同步号 SYN 置为 1，选择初始序号 $seq=x$ ，将该数据报发送给服务器，客户端进入 SYN_SENT 状态，等待服务器确认；

第二次握手：服务器收到数据报后，由 $SYN=1$ 知道客户端请求建立连接，将标志位 SYN 和 ACK 都置为 1，ack 设为 $x+1$ ，并选择初始序号 y ，将该数据报发送给客户端以确认连接请求，服务器进入 SYN_RCVD 状态；

第三次握手：客户端收到确认后，检查 ACK 是否为 1 以及 ack 是否为 $x+1$ ，如果正确则将 ACK 设为 1，ack 设为 $y+1$ ，将该数据报发送给服务器，服务器检查如果 $ACK=1$ ， $ack=y+1$ 则建立连接成功，客户端和服务器都进入 ESTABLISHED 状态，完成三次握手，

开始数据传输。



4.1 第一次握手

TCP 连接第一次握手，客户端发送请求信息，请求建立 TCP 连接，发送一个序列号 (SYN) 到服务器。如图 4.1.1 所示，SYN 为 1，序号为 2881245444。

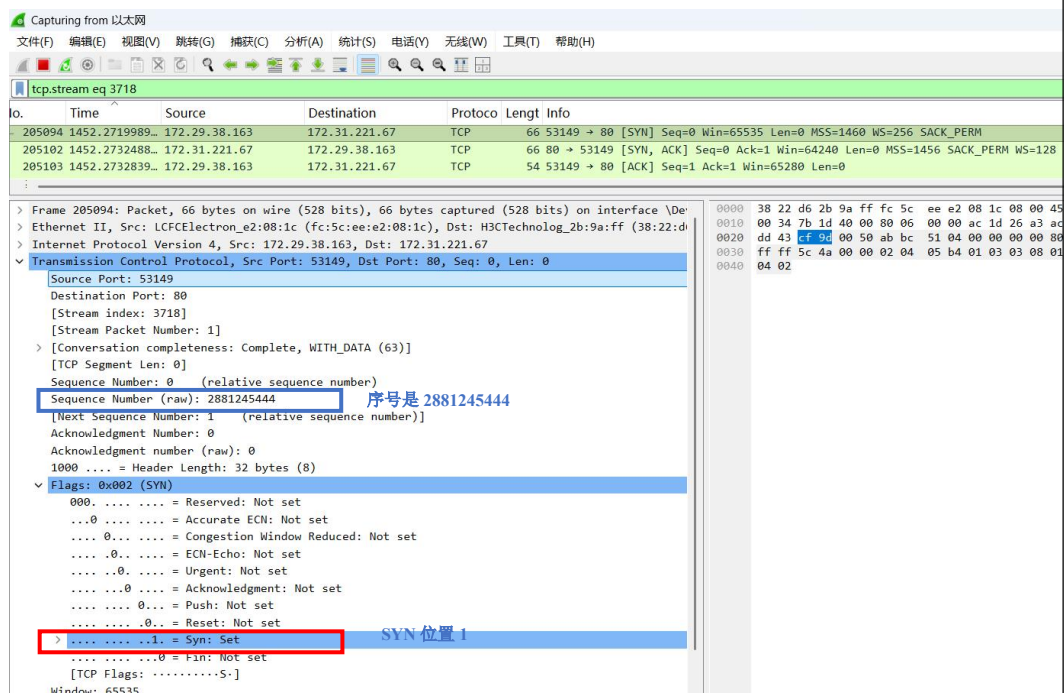


图 4.1.1 分析 TCP 三次握手，第一次握手 (SYN)

4.2 第二次握手

服务器收到信息后，返回一个 ACK=1 确认收到客户端信息，其确认号应该为 $x+1$ 。如图 4.2.1 所示，和图 4.1.1 相比，ACK 从 not set 变为 set，确认号为图 4.1.1 中序号+1。

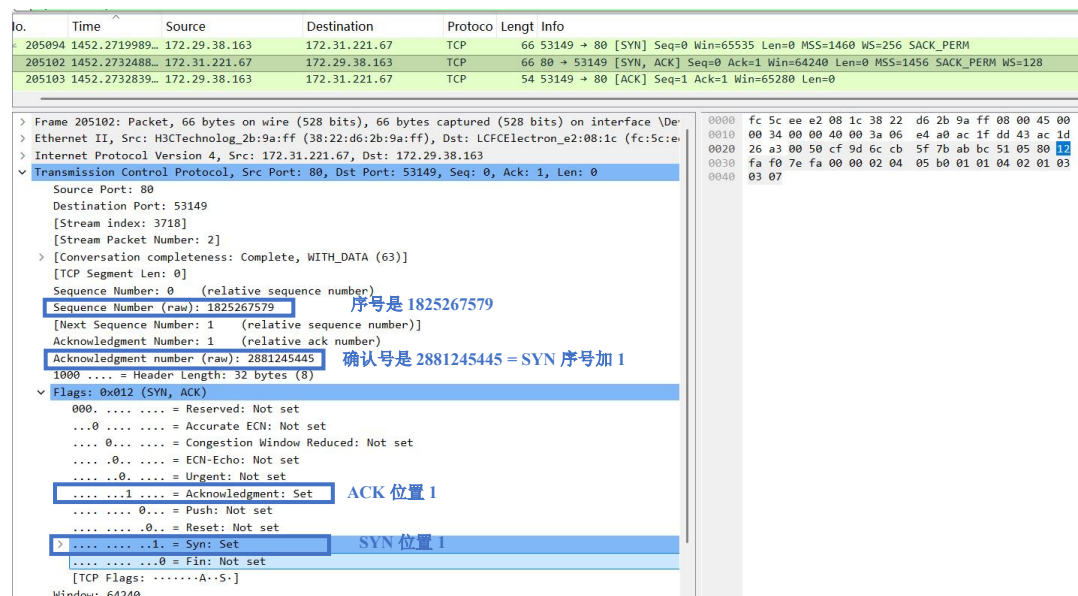


图 4.2.1 分析 TCP 三次握手，第二次握手 (SYNACK)

4.3 第三次握手

第三次握手时客户端向服务器发送一个 ACK，并发送确认号 $y+1$ ，说明客户端确认和

服务器建立 TCP 连接。如图 4.3.1 所示，客户端（端口：60537）向服务器（端口：80）发送的 ACK=1，确认号 2709081734 为图 4.2.1 的序号+1，第三次握手成功后即可建立连接，下面就是 HTTP 的 GET 方法了。

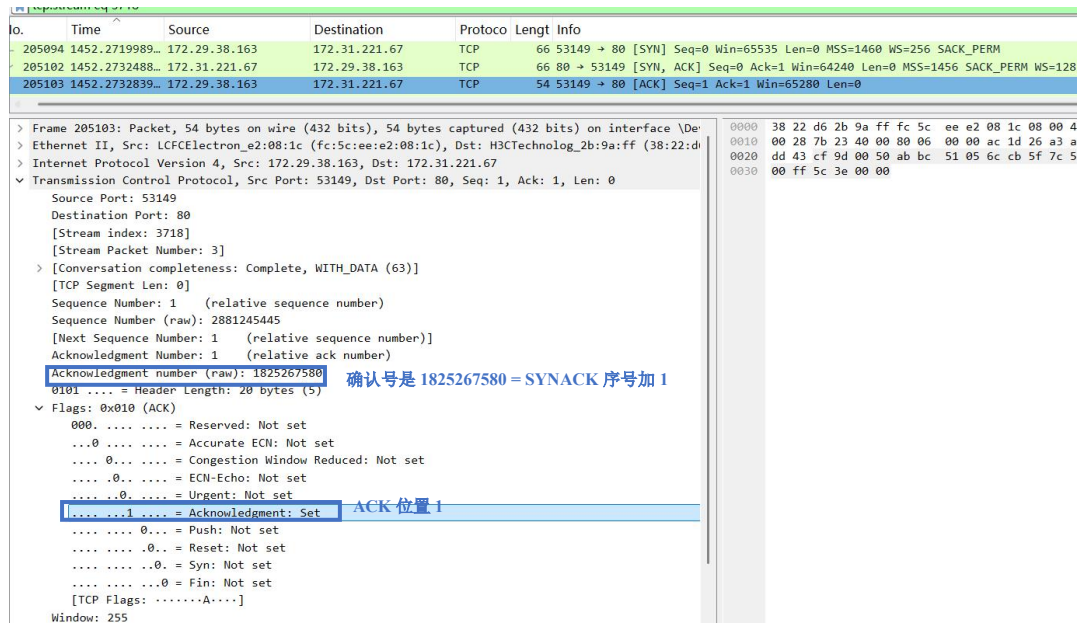


图 4.3.1 分析 TCP 三次握手，第三次握手（ACK）

5 分析 ICMP 协议

ICMP 协议（Internet Control Message Protocol）是一个网络层协议，一个新搭建好的网络，往往需要先进行一个简单的测试，来验证网络是否畅通；但是 IP 协议并不提供可靠传输，如果丢包其不能通知传输层是否丢包以及其原因。这时就需要一种协议来完成这样的功能——ICMP 协议。

在 Wireshark 过滤栏里输入 icmpv6 (这是 ping 指令使用的协议)，然后在 Powershell 里使用 ping 指令，得到的结果如图 5.1 所示。可以看到 ping 后 Wireshark 分组列表栏出现了被 ping 网址（百度）的 IP，协议为 ICMP，内容为 ping 指令的请求与回答。当没有丢失时，request 与 reply 均为默认的 4 组。

```
C:\Users\姚泽棉、>ping www.baidu.com
```

```
正在 Ping www.a.shifen.com [2409:8c54:870:310:0:ff:b0ed:40ac] 具有 32 字节的数据:
```

```
来自 2409:8c54:870:310:0:ff:b0ed:40ac 的回复: 时间=12ms
```

```
来自 2409:8c54:870:310:0:ff:b0ed:40ac 的回复: 时间=12ms
```

```
来自 2409:8c54:870:310:0:ff:b0ed:40ac 的回复: 时间=12ms
```

```
来自 2409:8c54:870:310:0:ff:b0ed:40ac 的回复: 时间=12ms
```

```
2409:8c54:870:310:0:ff:b0ed:40ac 的 Ping 统计信息:
```

```
数据包: 已发送 = 4, 已接收 = 4, 丢失 = 0 (0% 丢失),
```

```
往返行程的估计时间(以毫秒为单位):
```

```
最短 = 12ms, 最长 = 12ms, 平均 = 12ms
```

```
C:\Users\姚泽棉、>
```

icmpv6									
No.	Time	Source	Destination	Protocol	Length	Info			
2172	263.265331600	2001:250:3c00:3508::	ff02::1:ffff:ed49	ICMPv6	86	Neighbor Solicitation for 2001:250:3c00:3508:e0aa:d8f1:34fe:ed49 from 38:22:d6:2b:9a:ff			
2173	263.268900400	2001:250:3c00:3508::	ff02::1:ffff:ed49	ICMPv6	86	Neighbor Solicitation for 2001:250:3c00:3508:e0aa:d8f1:34fe:ed49 from 38:22:d6:2b:9a:ff			
2184	264.332361800	2001:250:3c00:3508::	ff02::1:ff17:8f87	ICMPv6	86	Neighbor Solicitation for 2001:250:3c00:3508:ac96:4d0f:ab17:8f87 from 38:22:d6:2b:9a:ff			
2189	265.334051200	2001:250:3c00:3508::	ff02::1:ff17:8f87	ICMPv6	86	Neighbor Solicitation for 2001:250:3c00:3508:ac96:4d0f:ab17:8f87 from 38:22:d6:2b:9a:ff			
3448	315.118040900	2001:250:3c00:3508::	ff02::1:ffa4:296e	ICMPv6	86	Neighbor Solicitation for 2001:250:3c00:3508:b16a:f4a8:32a4:296e from 38:22:d6:2b:9a:ff			
3814	352.323514300	2409:8c54:870:187:0::	2001:250:3c00:3508::	ICMPv6	94	Echo (ping) request id=0x0001, seq=761, hop limit=128 (reply in 3815)			
3815	352.334867900	2001:250:3c00:3508::	2409:8c54:870:187:0::	ICMPv6	94	Echo (ping) reply id=0x0001, seq=761, hop limit=49 (request in 3814)			
3821	353.339337500	2409:8c54:870:187:0::	2001:250:3c00:3508::	ICMPv6	94	Echo (ping) request id=0x0001, seq=762, hop limit=128 (reply in 3822)			
3822	353.351048500	2409:8c54:870:187:0::	2001:250:3c00:3508::	ICMPv6	94	Echo (ping) reply id=0x0001, seq=762, hop limit=49 (request in 3821)			
3827	354.347403700	2001:250:3c00:3508::	2409:8c54:870:187:0::	ICMPv6	94	Echo (ping) request id=0x0001, seq=763, hop limit=128 (reply in 3828)			
3828	354.358053900	2409:8c54:870:187:0::	2001:250:3c00:3508::	ICMPv6	94	Echo (ping) reply id=0x0001, seq=763, hop limit=49 (request in 3827)			
3838	355.358155700	2001:250:3c00:3508::	2409:8c54:870:187:0::	ICMPv6	94	Echo (ping) request id=0x0001, seq=764, hop limit=128 (reply in 3839)			
3839	355.369434500	2409:8c54:870:187:0::	2001:250:3c00:3508::	ICMPv6	94	Echo (ping) reply id=0x0001, seq=764, hop limit=49 (request in 3838)			
3848	355.915393900	2001:250:3c00:3508::	ff02::1:ffad:3bbd	ICMPv6	86	Neighbor Solicitation for 2001:250:3c00:3508:90bf:2b7b:ead3:bbd from 38:22:d6:2b:9a:ff			
4047	378.963496600	fe80::f124:9666:b9a...	fe80::3a22:d6ff:fe2...	ICMPv6	86	Neighbor Solicitation for fe80::3a22:d6ff:fe2b:9aff from fc:5c:ee:e2:08:1c			
4048	378.964929500	fe80::3a22:d6ff:fe2...	fe80::f124:9666:b9a...	ICMPv6	78	Neighbor Advertisement fe80::3a22:d6ff:fe2b:9aff (rtr, sol)			
4100	383.535155800	fe80::3a22:d6ff:fe2...	fe80::f124:9666:b9a...	ICMPv6	86	Neighbor Solicitation for fe80::f124:9666:b9ac:7bd3 from 38:22:d6:2b:9a:ff			
4101	383.535554100	fe80::f124:9666:b9a...	fe80::3a22:d6ff:fe2...	ICMPv6	86	Neighbor Advertisement fe80::f124:9666:b9ac:7bd3 (sol, ovr) is at fc:5c:ee:e2:08:1c			
> Frame 3821: Packet, 94 bytes on wire (752 bits), 94 bytes captured (752 bits) on interface \Device\NPF{...}							0000	38 22 d6 2b 9a ff fc 5c ee e2 08 1c 86 dd 60 00	8"-+...\\
> Ethernet II, Src: LCFCElectron_e2:08:1c (fc:5c:ee:e2:08:1c), Dst: H3CTechnolog_2b:9a:ff (38:22:d6:2b:9a:ff)							0010	00 00 00 28 3a 80 20 01 02 50 3c 00 35 08 70 2a	...{:...
> Internet Protocol Version 6, Src: 2001:250:3c00:3508:702a:f452:ebbc:2d48, Dst: 2409:8c54:870:187:0::							0020	f4 52 eb bc 2d 48 24 09 8c 54 08 70 01 87 00 00	-R...H\$-
> Internet Control Message Protocol v6							0030	00 ff b0 d9 bb 1c 80 00 99 d8 00 01 02 fa 61 62	
							0040	63 64 65 66 67 68 69 6a 6b 6c 6d 6e 6f 70 71 72	cdefghij
							0050	73 74 75 76 77 61 62 63 64 65 66 67 68 69	stuvwabc

图 5.1 对比 Wireshark 分组列表栏与 ping 指令显示结果

实验部分结束。

实验结果:

(给出个人对结果的分析、结论)

通过本次使用 Wireshark 进行数据提取与分析的实验,我得出了以下实验结果与结论:

1. HTTP 协议分析结论

明文传输特性:通过过滤 http 协议,可以清晰地捕获到浏览器向目标服务器(如 ieeexplore.ieee.org)发送的 GET 请求在分组详情栏,能够直接阅读到 HTTP 请求的头部信息,包括主机名(Host)、用户代理(User-Agent)以及接受的数据格式等。这仔细验证了 HTTP 协议在应用层是明文传输的。

IP 地址识别:通过分析捕获的数据包,能够准确提取出本地主机的本地源 IP 地址和目标服务器的公网目的 IP 地址。

2. TCP 协议与数据段分析结论

报文结构验证:在传输层,追踪特定的 TCP 流(如 tcp.stream eq 17)可以详细解析 TCP 报文段的内部结构实验结果成功提取了源端口号、目的端口号、相对序列号、确认号以及窗口大小等关键字段。

```
> Frame 205407: Packet, 724 bytes on wire (5792 bits), 724 bytes captured (5792 bits) on inter
> Ethernet II, Src: LCFCElectron_e2:08:1c (fc:5c:ee:e2:08:1c), Dst: H3CTechnolog_2b:9a:ff (38:
> Internet Protocol Version 4, Src: 172.29.38.163, Dst: 172.31.221.67
v Transmission Control Protocol, Src Port: 53149, Dst Port: 80, Seq: 6902, Ack: 2656, Len: 670
  Source Port: 53149      源端口号
  Destination Port: 80    目标端口号
  [Stream index: 3718]
  [Stream Packet Number: 33]
  > [Conversation completeness: Complete, WITH_DATA (63)]
  [TCP Segment Len: 670]
  Sequence Number: 6902 (relative sequence number)  序列号
  Sequence Number (raw): 2881252346
  [Next Sequence Number: 7572 (relative sequence number)]
  Acknowledgment Number: 2656 (relative ack number)
  Acknowledgment number (raw): 1825270235  确认号
  0101 .... = Header Length: 20 bytes (5)  报头长度
  > Flags: 0x018 (PSH, ACK)  标志位
  Window: 251  窗口大小
  [Calculated window size: 64256]
  [Window size scaling factor: 256]
  Checksum: 0x5edc [unverified]  校验和
  [Checksum Status: Unverified]
  Urgent Pointer: 0
  > [Timestamps]
  > [SEQ/ACK analysis]
  [Client Contiguous Streams: 1]
  [Server Contiguous Streams: 1]
  TCP payload (670 bytes)  数据
```

3. TCP 三次握手过程分析结论

TCP 连接的建立严格遵循三次握手机制,且该过程必然发生在 HTTPGET 请求发送之前。

第一次握手(SYN):客户端向服务器发送连接请求,标志 SYN 位置为 1,ACK 为 0,并生成一个初始序号 seq=0(相对序号)。

第二次握手(SYN-ACK):服务器收到请求后回复确认,标志位 SYN 和 ACK 均置为 1,确认号 ack=1(即客户端的 seq+1),并生成自己的初始序号 seq=0。

第三次握手(ACK):客户端再次确认,标志 ACK 位置为 1,SYN 置位为 0,序号等于 seq=1,确认号 ack=1(即服务器的 seq+1)。

结论:三次握手通过序列号和确认号的严密递增与交互,保证了通信双方的接收和发送能力均正常,从而建立了可靠的连接。

4. ICMP 协议分析结论

请求与应答匹配:通过在 PowerShell 中执行 ping 命令(如 ping www.bing.com),并在

Wireshark 中过滤 icmp 协议，可以观察到终端输出的 4 次数据包发送与接收，完美对应了 Wireshark 中捕获到的 4 对 Echo (ping) request 和 Echo (ping) reply 报文。

结论：Ping 命令本质上是基于 ICMP 协议运行的，通过发送回显请求和接收回显应答来测试网络初始化性并给出往返时间（RTT）。

实验小结：

1. 实验中遇到的问题及解决方法

问题一：后台流量庞杂，难以定位目标数据包。

解决方法：用 Wireshark 的过滤器语法，例如使用 `http` 过滤应用层协议，使用 `ip.src==xxx and ip.dst==xxx` 精确锁定的通信双方，以及利用正确的“追踪 TCP 流”功能。

问题二：icmp 过滤后没有数据。

解决方法：用 `icmpv6` 过滤，因为环境是 `ipv6`

问题三：HTTP 协议网站目前已经比较罕见，主流网页全部使用 HTTPS 协议加密过了

解决方法：寻找自己搭建的网站，如学校的 OJ

2. 实验心得体会 这次数据包提取与分析实验巩固了我的专业基础，让我真切地体验到了 TCP/IP 分层模型和三次握手机制，使我受益匪浅。

指导教师批阅意见：

成绩评定：

指导教师签字：

年 月 日

备注：